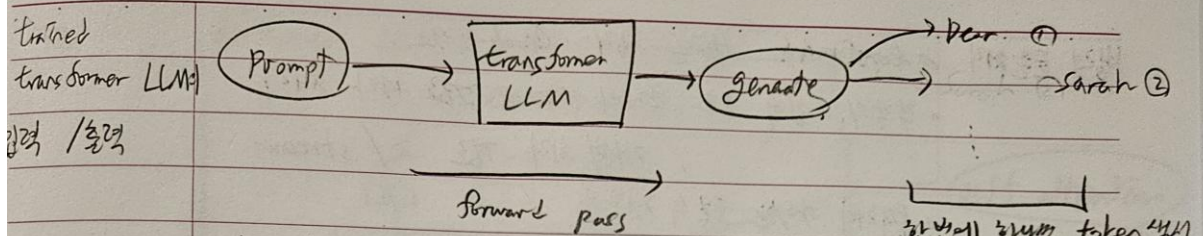
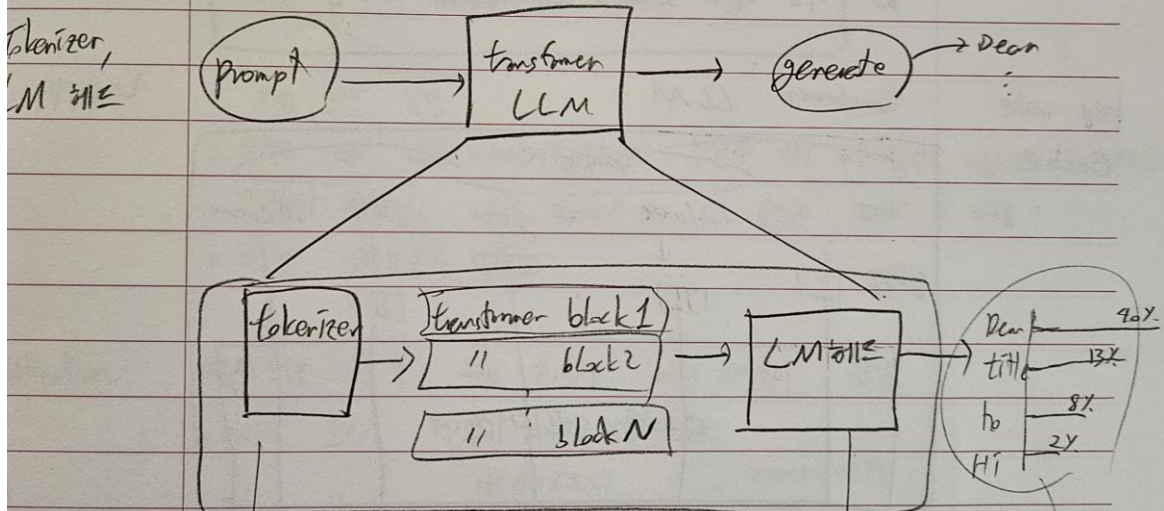


Transformer

PAGE
DATE



- 하나의 토큰 생성, 출력 토큰을 입력 prompt에 추가
- 입력 prompt 기반으로 다음 토큰 예측
- Autoregressive Model : 모델이 첫번째로 생성한 토큰을 사용하여 두번째 토큰 생성
! 텍스트 생성 모델 Vs BERT 같은 텍스트 표현 모델 구분 사용



→ 5000개에 구성된 여러 사전 가짐

→ 각 토큰에 연관된 토큰 embedding 가짐

각 토큰에 대한 확률 분포 출력 = 신경망층

- decoding (sampling)
- 여기서 하나의 토큰을 선택하는 전략, but 대부분은 가장 높은 확률 X
 - 약간의 무작위성을 가미하거나 두번째/세번째로 확률 높은 토큰 고르거나 이 방법
 - Greedy decoding : 항상 가장 높은 점수의 토큰 선택 방법
! LLM의 Temperature = 0

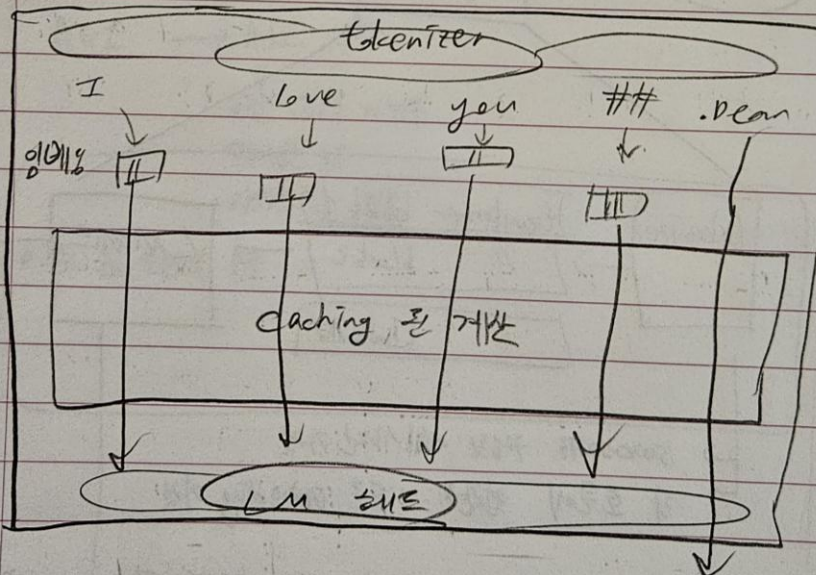
ex) 75 : 1m-heads output [0, -1] : batch size = 0
마지막 토큰 얻기 위해 인덱스 -1 사용

YANGJ1

- 병렬 토큰 처리
- Transformer는 병렬 처리 Good
 - 각각의 입력 토큰: 각자의 계산 경로 따라 처리
: 개별 처리 경로 / stream
 - 동시에 가능한 토큰 수 제한됨 =
모델의 문맥 길이 ex) 문맥 길이가 4K → 최대 4K개의 토큰만 처리 가능
 - 마지막 스트림의 출력 결과만 사용해 다음 토큰 예측
→ LM 헤드에 전달됨
 - 마지막 토큰 지시한 모든 토큰의 경로를 버림
but 이런 토큰의 stream이 최종 stream 계산에 쓰임

key value
cache

Transformer LLM



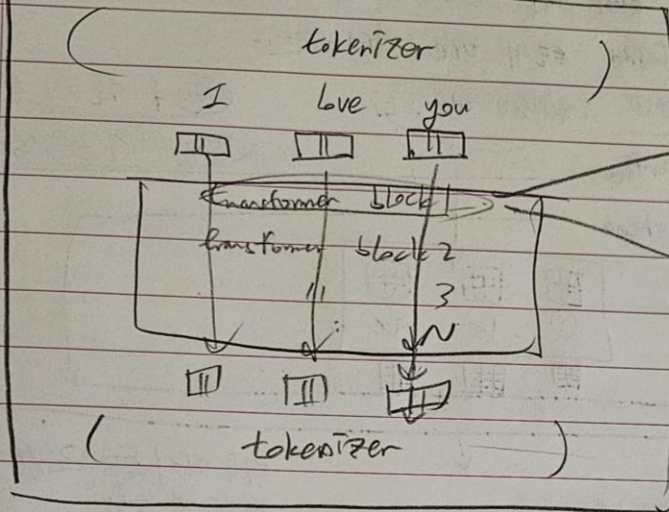
- 이런 계산의 결과 캐싱: 이런 stream의 계산 반복할 필요 X
- 마지막 stream에 대한 계산만 필요
- 하이퍼파라미터의 Transformer는 기본적으로 캐싱 사용
- 이런 최적화 기법: key value cache
- 첫 번째 출력 토큰은 입력에 덧붙여 또 한번 모델의 forward 계산 수행

transformer 내부

PAGE
DATE

구조

transformer LLM



self attention

FFNN

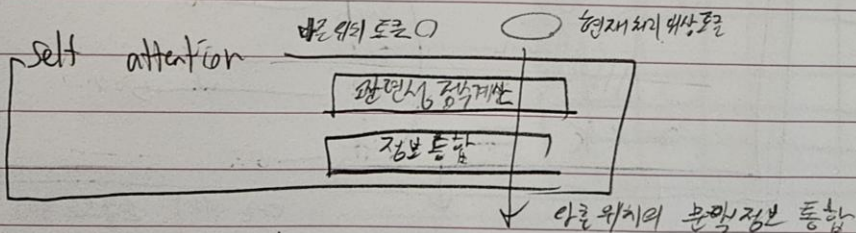
FFNN

- 기억, 보관 담당
- 대규모 text data 에서 성공적으로 훈련될 때 이 작업을 성공시키기 위한 정보 학습/저장
- 과거에 보지 않고 훈련 dataset 에 없는 문장에 대해 잘 수행
- 많은 정보량기 담당

Attention

문맥 정보 가 이 문장 안에서 어떤 의미인지 판단

- 마지막 토큰이 현재 처리되고 있는 토큰



- 이전 입력 토큰이 현재 처리 대상 토큰과 얼마나 관련 있는지 점수 매김
- 이걸 사용해 다양한 위치의 토큰에서 얻은 정보를 하나의 벡터에 통합

attention

- 병렬로 진행되는 각각의 어텐션 메커니즘

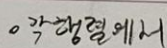
Head

- 입력 시퀀스에서 한번에 여러 패턴의 주의를 기울여야 하는 복잡한 패턴을 모델링
- 관련성 점수 계산하는 단계 / 어텐션 결과 통합 단계

이전 토론에 대한 벡터 표현

시퀀스의 다른 위치 찾기

현재 위치의 포큰점 



다른 위키 토크으로 부터
본인 정보 통합

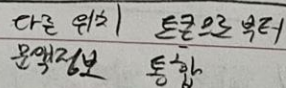
반 아래 행이 현재 위치의 로크에 해당

• 현재 위치 $Q \times K$

이후 softmax 연산을 통해 점수의 합이 1이 되도록 정규화

서클의 다른 위치 토큰
현재 위치의 토큰 값

현재 위치	도봉구
-------	-----



self attention

attention head

76 4181

X

[illegible]
$$\frac{1}{17} = \frac{1}{17}$$

110

IV 藝文

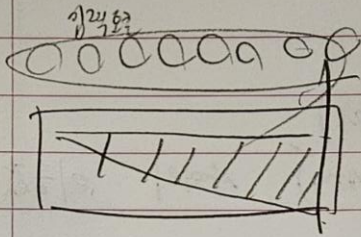
Transformer architecture 최근 발전

PAGE
DATE

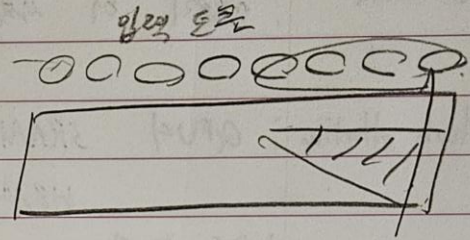
27월 / 1월
이벤션

Transformer 모델 ↑ → sparse attention
[sliding window attention 이
이벤션의 효율성을 개선

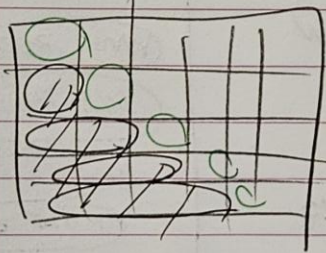
o 전역적인 self attention



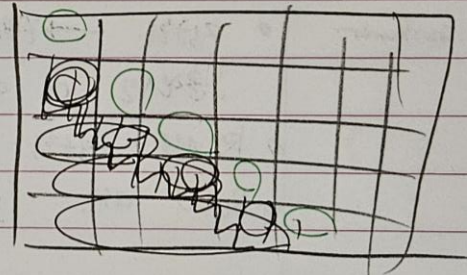
o Local self attention



o full attention

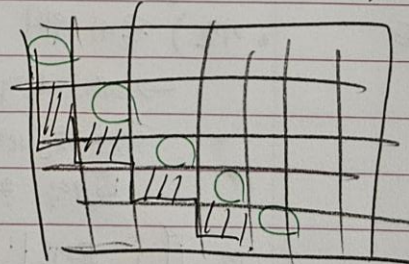


o sparse attention (stride)



o : 처리 중인 token
Q : 연관 있는 token

o sparse attention (226)



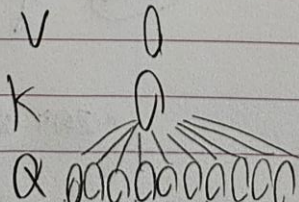
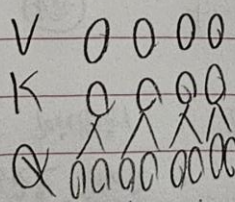
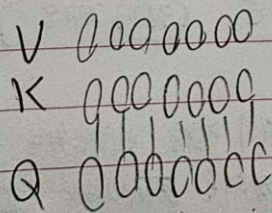
Multi Query
attention

Multi Head
Attention

Grouped Query
Attention

Multi Query
Attention

Grouped Query
attention

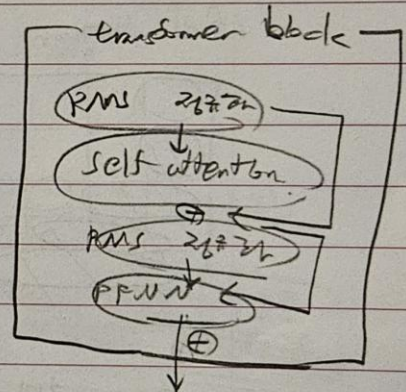


Multi Query Attention : K, V를 모두 head에서 공유
! 모델 크기 ↑ → 이런 최적화는 매우 과함.

Grouped Query Attention : K, V를 여러개 (head 개수보다는 적음)
! 어쨌든 더 사용.

Flash Attention : GPU의 SRAM (공유메모리)
HBM (고대역폭메모리) 사이의 이동량 최적화
→ 속도 높임.

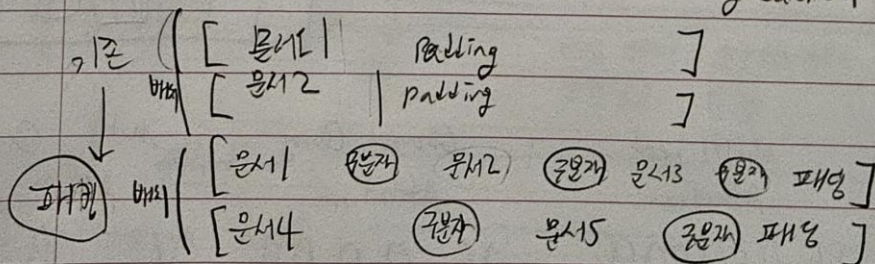
최신 transformer
• 정규화 → Attention, FFNN
! 훈련시킬 때 잔류시결
• RMS 정규화 사용
• ReLU 대신 SwiGLU 사용.



위치 임베딩 = Rotary Positional Embedding (RoPE)
• 기존) 절대 위치 임베딩 = 정적인 방법

→ 어려움 발생 : 확장성 개선 필요.

ex) Training set의 data가 AI model의 문장보다 짧다.
→ 모델 훈련시 문서 변경해 training batch에 문장 채움.



• Rotary 임베딩 : 상대적인 토큰 위치 정보를 인코딩하는 방법

! 임베딩 공간에 벡터를 회전시키는 Idea 기반

! Self Attention에서

! Q, K, V 공유하기 관련 특별한 해석으로 통합됨